stripped, so it is likely that the value you get back will have a trailing return or linefeed character.

```
`date`                 # => "Sun Dec 25 12:30:14 CST 2022\n"
`ls`.split[34]         # => "metaprogramming.md"
%x{echo "hello there"} # => "hello there\n"
```

You can use expression expansion and all the usual escape sequences in the command string:

```
0..3.each do |i|
  status = `dbmanager status id=#{i}`
  # ...
end
```

The exit status of the command is available in the global variable `$?`, also aliased as Process.last_status.

In the description of the command expression, we said that the string in backquotes would "by default" be executed as a command. In fact, the string is passed to the Kernel method called ` (a single backquote). If you want, you can override this method. This example uses Process.last_status, which contains the status of the last external process run:

```
alias old_backquote `

def `(cmd)
  result = old_backquote(cmd)
  unless Process.last_status.success?
    puts "*** Command #{cmd} failed: status = #{Process.last_status.exitstatus}"
  end
  result
end

print `ls -l /etc/passwd`
print `ls -l /etc/wibble`
```

produces:

```
-rw-r--r--  1 root  wheel  8160 Dec  2 05:37 /etc/passwd
ls: /etc/wibble: No such file or directory
*** Command ls -l /etc/wibble failed: status = 1
```

Assignment

Just about every example we've given so far in this book has featured assignment. It's about time we said something about it.

An assignment statement sets the variable or attribute on its left side (the *lvalue*) to refer to the value on the right (the *rvalue*). It then returns that rvalue as the result of the assignment expression. This means you can chain assignments, and you can perform assignments in some unexpected places:

```
a = b = 1 + 2 + 3
a         # => 6
b         # => 6
a = (b = 1 + 2) + 3
a         # => 6
b         # => 3
```